

Symbiosis Centre for Information Technology

(A Constituent of the Symbiosis International University, Pune)

AI Powered Developer Platforms and Tools

Submitted by

Name: Kunal Gupta

PRN: 26030242033

Batch: MBA-DSDA 2026-28

Date: 04.10.2026

Submitted to

Name of the Faculty : Dr. Yogesh Murumkar

Travel Policy Compliance Assistant

T26 — Chatbots & AI Assistants

1. Project Overview and Objective

The objective of this project was to develop an AI-assisted Travel Policy Compliance Assistant capable of reviewing employee travel requests against a defined set of company travel policies and providing a clear compliance decision.

The assistant classifies a travel request into one of three outcomes: APPROVED, REJECTED, or NEEDS APPROVAL. Along with the final decision, the system identifies the applicable policy rule and provides a reason for the decision.

The project was based on a dataset containing 150 travel requests and 15 travel policy rules. The travel-request dataset contains information such as employee name, grade, destination, trip type, request date, departure date, number of nights, travel mode, flight duration, hotel rate, purpose, and estimated total cost.

The policies cover areas including air travel, advance booking, train travel, hotel expenditure limits, daily allowance, local conveyance, alcohol reimbursement, expense receipts, claim deadlines, international travel, personal extensions, approval requirements, cancellation charges, and spouse/family travel.

The project was developed progressively through three major stages: a Python rule-based compliance engine, a terminal-based chatbot, and finally a browser-based chatbot built using Google AI Studio.

Hosted application: <https://t26-travel-policy-assistant.ai.studio/>

2. Approach and Methodology

2.1 Understanding and Translating the Policy

The first challenge was converting written business policies into structured decision logic. For example, hotel expenditure has to be compared with employee grade and whether the destination is considered metro or non-metro. Similarly, travel mode and flight duration have to be considered when evaluating air-travel eligibility.

The project therefore treated the policy rules as deterministic business rules rather than relying entirely on generative AI to make decisions.

2.2 Python Rule Engine

Python was used as the main data-processing layer. Pandas was used to load the Excel workbook and access the Travel Requests and Policy Rules sheets.

The program evaluated each request and returned a decision, rule ID, and reason. It processed all 150 requests and generated structured outputs for the decision sheet and the 30-request audit. The implementation also included logic to identify frequently triggered policy rules and an interactive terminal chatbot.

2.3 Debugging and Improving the Python Logic

The first version of the Python code did not produce reliable results. It used placeholder fields that did not correspond correctly to the actual workbook structure. As a result, the initial implementation incorrectly approved all 150 requests.

This demonstrated that AI-generated code can execute successfully while still being logically incorrect for a particular dataset. The code was subsequently revised using the actual travel-request columns and policy definitions.

A further issue involved TP-13. Treating every domestic trip as automatically requiring additional approval would cause almost every domestic request to be classified as NEEDS APPROVAL. The final hierarchy was therefore: policy violation → REJECTED; additional approval requirement → NEEDS APPROVAL; no violation/additional approval → APPROVED. Rules for which sufficient data was unavailable were treated as NOT TESTABLE.

2.4 Terminal-Based Chatbot

Once the Python rule engine was working, a simple conversational interface was added to the Python program. The user could enter a Request ID and receive the decision, rule ID, and reason directly through the terminal.

The terminal chatbot demonstrated that the compliance engine could be used interactively. However, it was not ideal as the final user-facing solution because it was less intuitive and did not provide a rich visual display of travel particulars, analytics, or policy checks.

3. Google AI Studio Web Application

3.1 Moving from Terminal to Web

To make the assistant more practical and presentation-ready, the project was converted into a browser-based application using Google AI Studio Build Mode.

The final application is hosted at <https://t26-travel-policy-assistant.ai.studio>. The web application provides a graphical interface instead of requiring the user to run Python commands or interact with a terminal. The main interaction is based around the travel Request ID.

A user can search for a request and receive its corresponding travel particulars and compliance result.

3.2 Using the Actual Travel Dataset

An important improvement was moving beyond generic chatbot responses and connecting the application to the actual travel-request data. The chatbot was modified to work with the actual travel requests rather than relying only on sample examples.

This allows the application to support Request IDs such as TR0001, TR0002, TR0003 and so on, and to display the corresponding travel particulars and compliance result.

3.3 User Interface Improvements

During testing, several practical UI issues were identified. When the application was viewed in full-screen mode on a laptop, the travel-request particulars were not being displayed properly after searching for a request. The dataset analytics section was also not functioning correctly.

The interface additionally contained two search boxes performing essentially the same Request ID search. The placeholder text used an example such as TR-101 even though the actual dataset uses the format TR0001, TR0002 and similar IDs. These issues were identified through testing and communicated to Google AI Studio for refinement.

4. Testing, Recommendations and Future Improvements

4.1 Testing Strategy

Testing was performed at multiple levels. At the data level, all 150 travel requests were processed by the Python rule engine. The objective was to ensure that every request received a decision and that the decision could be associated with a rule or explanation.

A sample of 30 requests was manually checked to compare automated decisions against the underlying policy. This was particularly important because the project had already demonstrated that an automated system can produce apparently valid output while using incorrect assumptions.

At the application level, the web chatbot was tested for Request ID lookup, travel particulars, compliance results, policy explanations, analytics, search functionality, full-screen laptop usability, Request ID format, and duplicate interface elements.

4.2 Recommendations

1. Maintain deterministic policy rules. The final compliance decision should remain based on explicit policy rules. Generative AI is better used for explaining decisions and interacting naturally with users.

2. Expand the dataset. Future versions should include fields for flight class, advance booking days, daily allowance, local conveyance, alcohol expenses, receipt availability, claim submission date, cancellation reason, personal extension, spouse/family travel, and approval status.

3. Add stronger data validation. The chatbot should validate Request ID format, dates, employee grade, destination, hotel rate, estimated cost, and travel mode.

4. Maintain an audit trail. A production version should record Request ID, timestamp, data used, rules evaluated, decision, reason, and policy version.

5. Improve analytics. Future analytics could show APPROVED vs REJECTED vs NEEDS APPROVAL, most frequently triggered rules, violations by employee grade and destination, average travel cost, high-cost travel requests, and requests requiring additional approval.

5. Reflections and Conclusion

This project significantly changed my understanding of how AI should be applied to business problems. Initially, I viewed the task primarily as a coding problem: read the Excel file, apply the policy rules, and generate a decision. However, the first implementation showed that technically executable code does not necessarily produce correct business results.

The initial implementation approved all 150 requests because it was based on incorrect assumptions about the dataset columns. This was one of the most important learning points from the project. It demonstrated the importance of validating AI-generated code against the actual business data.

I also learned that a compliance assistant needs to be explainable. A simple answer such as REJECTED is not enough for a business user. The user needs to know which rule was triggered and why.

The distinction between REJECTED, NEEDS APPROVAL, and NOT TESTABLE was another important learning. A missing data field should not automatically be treated as a policy violation. At the same time, it should not be silently ignored. Making uncertainty explicit through NOT TESTABLE provides a more responsible approach.

The project also showed the difference between a technical prototype and a usable application. The Python rule engine provided the core computational logic. The terminal chatbot demonstrated that this logic could be accessed interactively. The Google AI Studio web application then transformed that prototype into a more accessible interface suitable for an actual user.

The development therefore followed a progression: Excel Dataset → Python Rule Engine → Terminal Chatbot → Google AI Studio Web Chatbot.

Python solved the data-processing and policy-evaluation problem. The terminal interface solved the basic interaction problem. Google AI Studio solved the user-experience and web accessibility problem.

The project also reinforced the importance of iterative development. The final chatbot was not created perfectly in one attempt. Issues involving the dataset, travel-request display, analytics, duplicate search boxes, and Request ID formatting were discovered through actual use and then corrected.

From a business perspective, I would not recommend deploying the current system as a fully autonomous approval mechanism. Instead, I would position it as a decision-support assistant that performs the initial compliance screening and provides transparent reasoning, while final approval remains with the appropriate human authority.

Overall, the project demonstrated how AI, Python, structured business rules, and a web interface can be combined to solve a practical business-process problem.

The key learning from the project is: AI can accelerate the development of a business solution, but reliable results still depend on accurate data, clearly defined rules, testing, explainability, and human validation.

The final Travel Policy Compliance Assistant provides a working proof of concept and a foundation that could be extended into a more comprehensive enterprise travel-compliance system.

Live Application

<https://t26-travel-policy-assistant.ai.studio>